Final Project

Due Thursday, December 9
Last revised: September 29, 2022

For the final project, you will implement a code to solve the two-dimensional (2D), incompressible Navier-Stokes equations in a periodic domain. 2D turbulence is relevant to geophysical turbulence on large horizontal scales because of the thinness of Earth's atmosphere and Earth's rotation and stable stratification, both of which tend to suppress vertical flow and make the 2D horizontal velocity component dominant. We will use a staggered second-order central finite-difference method and a third-order Runge-Kutta time advancement scheme, along with the fractional step method. Small portions of the project are to be completed as part of each homework set to ensure you are on track to finish the project by the end of the quarter.

DOMAIN AND NONDIMENSIONALIZATION

Let the x and y directions have extent $2\pi L_x$ and $2\pi L_y$. We will take all lengths to be normalized by L_x, velocities normalized by a reference speed U_0, and time normalized by L_x/U_0. Other quantities are nondimensionalized consistently, with respect to these reference quantities. We will use N_x and N_y uniformly distributed grid points in space to divide up our domain.

The boundary condition for this project is periodic. This means that the first grid point is connected to the last grid point (think of an array of tiles on the bathroom floor, where each tile is our domain). Normally, the grid point i is adjacent to $i-1$ and $i+1$. For grid point N_x in the x direction, this grid point is adjacent to $N_x - 1$ and 1. The same applies to the y direction.

The nondimensional incompressible Navier-Stokes equation is given by

$$\frac{\partial u}{\partial t} + \frac{\partial uu}{\partial x} + \frac{\partial uv}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{Re}\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) \tag{1}$$

$$\frac{\partial v}{\partial t} + \frac{\partial uv}{\partial x} + \frac{\partial vv}{\partial y} = -\frac{\partial p}{\partial y} + \frac{1}{Re}\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) \tag{2}$$

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{3}$$

where u and v are the velocity components in the x and y directions, $Re = U_0 L_x/\nu$ is the Reynolds number, and ν is the kinematic viscosity.

TIME ADVANCEMENT

For the project, we'll choose a third-order Runge-Kutta (RK3) scheme, which will work well for this problem. The Butcher Tableau for a RK3 scheme (not unique) is given in Table 1.

As discussed in class, there will be stability limits on the time step Δt for RK3. These stability limits are difficult to determine for general equations, but as we discussed in class, estimates for the stability limits can be obtained by the Courant–Friedrichs–Lewy (CFL) condition as

$$\Delta t_{\max} = \min\left(\frac{C\Delta_x}{\max(u)}, \frac{C\Delta_y}{\max(v)}\right),$$

0	0	0	0
1/2	1/2	0	0
1	-1	2	0
	1/6	2/3	1/6

Table 1: Butcher Tableau for a third-order Runge-Kutta scheme.

where C is the Courant number, Δ_x and Δ_y are the grid resolutions in each direction. For RK3, $C \approx 1.75$, but more commonly, $C = 1$ is used to make sure the simulation is stable.

To be very safe, you could monitor the maximal velocity component and adjust the time step so that the conditions above are satisfied at every time step. However, if U_0 is wisely chosen, the maximum velocity $O(1)$ at any time. This will allow the time step to be fixed for all time and makes post-processing the results easier.

SPATIAL DISCRETIZATION

We will use a staggered second-order central finite-difference method. This means that the velocity components will be stored in different parts of the grid. The u component will be located on the x-faces of the grid, whereas the v component will be located on the y-faces of the grid. All other scalar values (e.g. pressure) will be located on the grid center. See figure 1 for a demonstration of where each component of the equation is stored.

This staggered discretization means that sometimes interpolation is necessary to ensure the components of each of the Navier-Stokes equations are evaluated at the same location. For example, in Eq. (1), all but the $\partial uu/\partial x$ term are naturally located on the x-faces of the grid, and thus $\partial uu/\partial x$ needs to be interpolated to those locations before being added to the rest of the terms. Using second-order interpolation, $\partial uu/\partial x$ can discretely be evaluated as

$$\begin{aligned}\frac{\delta uu}{\delta x}(i,j) &= \frac{1}{2}\left(\frac{\delta uu}{\delta x}(i-1/2,j) + \frac{\delta uu}{\delta x}(i+1/2,j)\right) \\ &= \frac{1}{2}\left(\frac{u(i,j)^2 - u(i-1,j)^2}{\Delta x} + \frac{u(i+1,j)^2 - u(i,j)^2}{\Delta x}\right).\end{aligned}$$

Additional interpolation is necessary for $\partial uv/\partial y$, for example, to make sure u and v are located at the same point when computing uv. The choice of interpolation is not unique, but using the minimum number of interpolations will improve the accuracy of your solution.

FRACTIONAL STEP (PROJECTION) METHOD

Looking at the Navier-Stokes equations, you may notice that there are three unknowns, u, v, and p. While there are equations to advance u and v in time, there is no equation advancing p in time. To overcome this issue, oftentimes fractional step methods are used. Fractional step (or projection) methods are a widely used technique for uncoupling the pressure solution in the incompressible Navier-Stokes equations while still satisfying the incompressibility constraint.

For a simple Euler time-advancement, the fractional step method at time step n will look like

$$\frac{u_n^* - u_n}{\Delta t} = -\frac{\delta u_n u_n}{\delta x} - \frac{\delta u_n v_n}{\delta y} + \frac{1}{Re}\left(\frac{\delta^2 u_n}{\delta x^2} + \frac{\delta^2 u_n}{\delta y^2}\right) \tag{4}$$

$$\frac{v_n^* - v_n}{\Delta t} = -\frac{\delta u_n v_n}{\delta x} - \frac{\delta v_n v_n}{\delta y} + \frac{1}{Re}\left(\frac{\delta^2 v_n}{\delta x^2} + \frac{\delta^2 v_n}{\delta y^2}\right) \tag{5}$$

$$\frac{u_{n+1} - u_n^*}{\Delta t} = -\frac{\delta p}{\delta x}, \quad \frac{v_{n+1} - v_n^*}{\Delta t} = -\frac{\delta p}{\delta y} \tag{6}$$

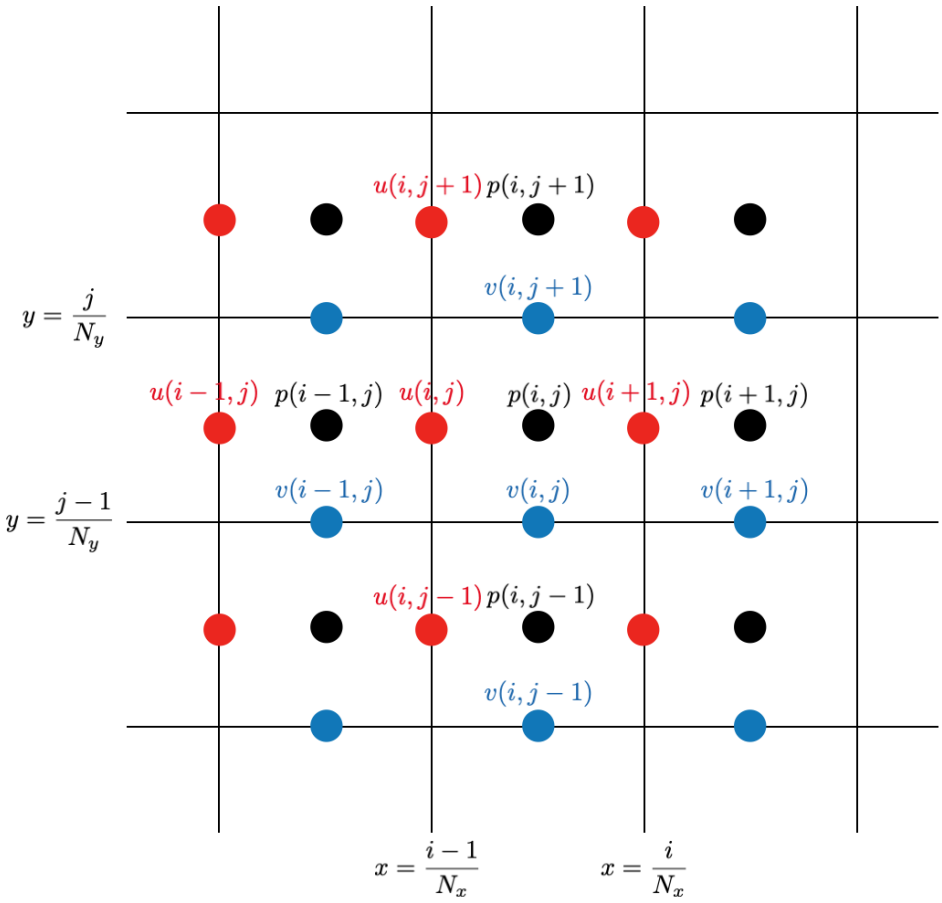


Figure 1: Location of the u (red), v (blue), p (black) variables in a staggered grid.

Note that the first two equations do not include p. However, the intermediate velocity u_n^* and v_n^* will not satisfy the incompressibility constraint. The value of p in the last two equations are then used to ensure u_{n+1} and v_{n+1} will satisfy continuity. This p is not necessarily the true value for pressure and is called pseudo-pressure for this reason.

Taking the gradient of the last two equations and adding them gives

$$\frac{\delta}{\delta x}\left[\frac{u_{n+1}-u_n^*}{\Delta t}\right]+\frac{\delta}{\delta y}\left[\frac{v_{n+1}-v_n^*}{\Delta t}\right]=-\left(\frac{\delta^2 p}{\delta x^2}+\frac{\delta^2 p}{\delta y^2}\right).$$

Since we want $\delta u_{n+1}/\delta x+\delta u_{n+1}/\delta y=0$, this leaves

$$\frac{\delta u_n^*}{\delta x}+\frac{\delta v_n^*}{\delta y}=\Delta t\left(\frac{\delta^2 p}{\delta x^2}+\frac{\delta^2 p}{\delta y^2}\right). \tag{7}$$

Solving the Poisson equation for p, we can substitute the p into Eq. 6 to obtain u_{n+1} and v_{n+1}. See the next section for details on the Poisson solver.

The explanation above was for a Euler time advancement. However, the same fractional step method works for RK3 as well. This is done by solving Eqs. (4) and (5), computing the pseudo-pressure from Eq. (7), and applying the projection step as in Eq. (6) at each RK3 step.

POISSON SOLVER

The Poisson equation

$$\nabla^2 p=\frac{\partial^2 p}{\partial x^2}+\frac{\partial^2 p}{\partial y^2}=f, \tag{8}$$

where in this case $f=\Delta t(\partial u/\partial x+\partial v/\partial y)$, can be solved using Fourier transformations and modified wavenumbers. Recall that

$$\mathcal{F}\left[\frac{\partial p}{\partial x}\right]=\mathrm{i}k_x\hat{p},$$

where $\mathrm{i}=\sqrt{-1}$, $\mathcal{F}$ or $(\hat{\cdot})$ is the Fourier transform, and k_x is the wavenumber. Because we are using a second-order finite-difference, for consistency with numerics, we use the modified wavenumber k_x' instead (you can check that divergence is not zero if k_x and k_z are used instead!). Thus, taking the Fourier transform of Eq. (8), we get

$$-(k_x'^2+k_z'^2)\hat{p}=\hat{f}$$

or

$$\hat{p}=-\frac{1}{k_x'^2+k_z'^2}\hat{f}.$$

Caution is necessary for $k_x'=k_z'=0$, as this will result in division by zero. In solving Poisson equations, the value of $\hat{p}(0,0)$ will typically be set by the boundary condition. In this case, since p is used as an intermediary variable, and the absolute value of p is not necessary (only the derivatives are needed to update Eq. (6)), we can set $\hat{p}(0,0)$ to an arbitrary value.

IMPLEMENTATION

1. To setup the problem, you will need to define the global variables that can be accessed from all components of the code. These could include the x and y coordinates, Δx and Δy, N_x and N_y, the Reynolds number, etc. Note that since the domain is normalized $0\le x\le 2\pi$ and $0\le y\le 2\pi$. You will also need to set up a function `main` that will define these global variables, setup an initial condition, and run the time-loop.

2. For the first component of the code, you will implement the RK3 scheme for a general ODE or PDE,

$$\frac{du}{dt} = F_u, \quad \frac{dv}{dt} = F_v.$$

Write a function or module named time_advance_RK3 that takes as input u, v, and Δt and outputs the u and v at the next time step. Use a second function named compute_RHS, which takes as input u and v and returns the the right-hand-side of the ODE or PDE you are evaluating (i.e., F_u and F_v). Eventually, this will compute the RHS of the Navier-Stokes equations; however, you will want to test your RK3 scheme using simpler functions at first.

After writing the time-advancement scheme, test your code using model problems with analytic solutions (we went through many examples in class and homework). This will allow you to make sure the RK3 scheme is third-order accurate. Try several different time-steps and draw an error convergence diagram to show the convergence.

3. For the second part of the project, you will implement the staggered second-order finite-difference scheme for Eqs. (4) and (5) for the compute_RHS function. The function will take as input u and v and return the RHS F_u and F_v. Take particular care in making sure all the terms involved are evaluated at the correct location.

Test this function by assigning a periodic function with an analytic form as input (think sin and cos), and making sure the RHS has the correct convergence behavior. Using several different grid resolutions, make sure the code is indeed second-order accurate.

4. For the final part of the project, you will implement the fractional step method. This involves taking as input u and v, computing the pseudo-pressure p by solving Eq. (7), and outputting the divergence-free u and v using Eq. (6). Write a function compute_projection_step that does this. This will involve i) computing the RHS of Eq. (8) from u and v (again, make sure things are evaluated at the correct location!), ii) taking the Fourier transform, iii) solving for $\hat{p}$, iv) taking the inverse Fourier transform, v) then updating u and v using Eq. (6). You will notice that although Δt is present in these equations, you won't actually need Δt for this process as they cancel out. Also, since the modified wavenumbers only need to be computed once, consider defining them as global variables to be initialized in main.

Verify this portion of the code by inputting a randomly generated u and v and making sure the result is divergence free. If everything is well implemented, the divergence should be $O(10^{-15})$.

The compute_projection_step function should be called after each compute_RHS evaluation in time_advance_RK3. And now you have a working code for solving the 2D incompressible Navier-Stokes equations!

5. Now that you have a working code, you can actually test if all the components are working together. One great case for validation is the Taylor-Green vortex solution. The 2D Taylor-Green vortex has an analytic solution for the incompressible Navier-Stokes equations that satisfy

$$u = \sin(x)\cos(y)e^{-2t/Re}$$
$$v = -\cos(x)\sin(y)e^{-2t/Re}.$$

Compare your numerical solution to the analytical solution after $T = 0.1$ with three grid resolutions $N_x = N_y = 64, 128, 256$ (for fixed Re) and show the accuracy of the code.

You should be able to now determine the empirical stability bounds of your time marching scheme. Initialize the velocity field to randomly generated u and v. Use various values of Δt and Re to empirically check the time step stability. Check if the stability depends on Re.

Finally, you will need a way to visualize the flow field. This could involve saving the flow field and later post-processing the data, or outputting a image file during the simulation.

PROJECT SUBMISSION

Now that you have the basic implementation of the 2D solver, the main portion of the project is to either 1) improve your code (e.g., extend it to 3D, implement higher-order methods, parallelize your code, etc.) or 2) perform some physical experiments with your developed code (e.g., run an isotropic turbulence and plot the change in energy spectra in time, run a mixing layer simulation and plot the mixing length as a function of time, etc.). The specifics of the project is up to you. Feel free to discuss the idea for your project beforehand.

You will need to submit a **short report** on the implementation of your code as well as documentation on the improvement/physical experiment of your choice. In addition, you will need to submit a **visualization (video)** of a simulation you ran using your code.

Do not submit your code unless necessary!